Ex. No.:6	AI-POWERED MEDICAL SYMPTOM ASSISTANT USING GPT-4	REG NO: URK23CS1033
DATE: 04/11/2026		

Aim:

To develop an AI-powered medical symptom assistant using GPT-4 that analyzes user-reported symptoms, identifies possible associated conditions, generates relevant follow-up questions and provides general guidance regarding professional medical attention.

Problem Statement:

Design and develop an AI-powered medical symptom assistant using GPT-4. The application should collect basic user information and symptoms, analyze the reported symptoms, identify possible conditions, generate follow-up questions and present the results in a structured format. The system should include appropriate safety instructions and clearly state that the generated output is not a medical diagnosis.

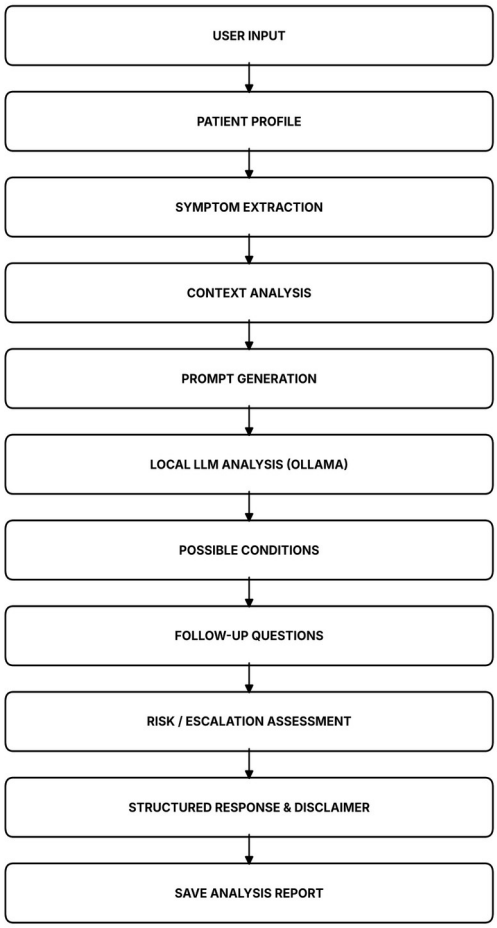
Software Requirements:

Software	Version
Python	3.11.15
Ollama	0.23.1
Llama 3.2 (local LLM used in place of GPT-4)	llama3.2:latest
Pandas	3.0.5
Sentence Transformers	6.0.1
Cursor (VS Code based editor)	3.15.19

Input Parameters

Parameters	Description
Age	Age of the person reporting the symptoms. Checked for a numeric value under 120.
Sex	Sex, where relevant to the reported symptoms.
Symptoms	Free natural-language description of the symptoms, entered in the user's own words.
Duration	How long the symptoms have been present.
Severity	Mild, Moderate or Severe, as reported by the user.
Medical History	Previously known relevant conditions.
Medication	Medications currently being taken, if relevant.
Allergies	Known allergies, if relevant.
Additional Information	Any other detail the user wishes to add.

System Architecture



System Architecture of the AI-Powered Medical Symptom Assistant

Module Design

Module	**Responsibility**
Input Module	Collects the age, sex, duration, severity, medical history, medication, allergies and additional information, together with the natural-language symptom description, and displays the symptom input.
Profile Module	Normalises the collected fields into a patient profile, checks the age and severity values and displays the patient information.
Symptom Module	Uses the local model to extract the individual symptoms from the natural-language description and cleans the extracted list.
Context Module	Lists the contextual fields the user did not provide and uses Sentence Transformers to find the closest reference cases, reporting the details usually relevant for symptoms of that kind.
Prompt Module	Builds the analysis prompt with the patient information, the extracted symptoms and the missing information, and states the safety rules the model must follow.
Analysis Module	Sends the prompt to the local model, splits the response into its four sections, displays the possible conditions and assembles the complete structured response.

Module	Responsibility
Question Module	Generates five short follow-up questions from the symptoms, the missing information and the details usually checked for similar cases.
Safety Module	Produces the medical attention guidance, flags wording that is generally treated as urgent and displays the required disclaimer.
Save Module	Writes the structured analysis and the raw model response to outputs/medical_analysis.txt.
Main Module	Provides the nine-option menu and connects all the modules into a single application.

Implementation

Patient information input (input_module.py)

```
    def collect():
        """Q1 - collect the patient information and the symptom description."""
        print("Enter the patient information. Press Enter to skip a field.")
        print(utils.DASH)

        record = {}
        for key, label, default in FIELDS:
            value = input(label).strip()
            record[key] = value or default

        print(utils.DASH)
        print("Describe the symptoms in your own words.")
        record["symptom_text"] = input("Symptoms                : ").strip()

        if not record["symptom_text"]:
            print("A symptom description is required.")
            return None

        utils.update_state(**record)
        return record
```

Symptom extraction (symptom_module.py)

```
    def extract(verbose=True):
        """Q2 - extract the symptoms from the natural-language input."""
        text = utils.require("symptom_text", "Please enter the symptoms first (option 1).")
        if not text:
            return None

        raw = utils.ask(SYSTEM, "Patient description:\n{}\n\nList the symptoms.".format(text),
                        temperature=0.1)
        symptoms = clean_list(raw)
        utils.update_state(symptoms=symptoms, symptom_count=len(symptoms))

        if verbose:
            print("Extracted Symptoms")
            print(utils.DASH)
            print("Patient description:")
            for line in utils.wrap(text, 58):
                print("  {}".format(line))
            print(utils.DASH)
            print("Symptoms identified by the model:")
            for item in symptoms:
                print("  * {}".format(item))
            print(utils.DASH)
            print("Model            : {}".format(utils.OLLAMA_MODEL))
            print("Symptoms found   : {}".format(len(symptoms)))
            print(utils.DASH)
        return symptoms
```

Context analysis (context_module.py)

```
    def missing_fields(state):
        """List the contextual fields the user has not supplied."""
        return [label for key, label in CONTEXT_FIELDS
                if str(state.get(key, "")).strip().lower() in EMPTY]


    def similar_cases(symptom_text, top=3):
        """Find the reference cases whose symptoms are closest to the description."""
```

```
    from sentence_transformers import util

    frame = utils.load_cases()
    model = _embedder()
    query = model.encode(symptom_text, convert_to_tensor=True)
    corpus = model.encode(frame["symptoms"].tolist(), convert_to_tensor=True)
    scores = util.cos_sim(query, corpus)[0]

    order = scores.argsort(descending=True)[:top]
    rows = []
    for idx in order:
        i = int(idx)
        rows.append({"Case": frame.at[i, "case_id"],
                     "Category": frame.at[i, "category"],
                     "Usual Duration": frame.at[i, "typical_duration"],
                     "Score": round(float(scores[i]), 4)})
    checks = []
    for idx in order:
        for item in str(frame.at[int(idx), "commonly_checked"]).split(","):
            item = item.strip()
            if item and item not in checks:
                checks.append(item)
    return rows, checks
```

Prompt generation (prompt_module.py)

```
def build_prompt(verbose=True):
    """Q4 - construct the analysis prompt for the model."""
    state = utils.load_state()
    if not state.get("symptom_text"):
        print("Please enter the symptoms first (option 1).")
        return None

    prompt = TEMPLATE.format(
        age=state.get("age", "Not provided"),
        sex=state.get("sex", "Not provided"),
        duration=state.get("duration", "Not provided"),
        severity=state.get("severity", "Not provided"),
        history=state.get("history", "None reported"),
        medication=state.get("medication", "None reported"),
        allergies=state.get("allergies", "None reported"),
        additional=state.get("additional", "None reported"),
        symptom_text=state.get("symptom_text", ""),
        symptoms="\n".join("- {}".format(s) for s in state.get("symptoms", [])) or "- not
extracted",
        missing="\n".join("- {}".format(m) for m in state.get("missing", [])) or "- none",
    )

    utils.update_state(prompt=prompt, prompt_chars=len(prompt))

    if verbose:
        print("Prompt Generation")
        print(utils.DASH)
        print("System role    : informational medical symptom assistant")
        print("Headings asked : Possible Conditions, Explanation,")
        print("                 Additional Information Needed, Medical Attention")
        print("Safety rules   : 6 (no diagnosis, no invented information)")
        print("Prompt chars   : {}".format(len(prompt)))
        print("Prompt words   : {}".format(len(prompt.split())))
        print(utils.DASH)
        print("Generated prompt (first lines)")
        print(utils.DASH)
        utils.preview(prompt, lines=22)
```

```
        print(utils.DASH)
    return prompt
```

Symptom analysis (analysis_module.py)

```
def analyze(verbose=True):
    """Q5 - generate the preliminary symptom analysis with the local model."""
    prompt = utils.load_state().get("prompt") or prompt_module.build_prompt(verbose=False)
    if not prompt:
        return None

    response = utils.ask(prompt_module.SYSTEM, prompt, temperature=0.2)
    sections = split_sections(response)
    utils.update_state(response=response, sections=sections,
                       response_words=len(response.split()))

    if verbose:
        print("Model Response")
        print(utils.DASH)
        print("Model          : {}".format(utils.OLLAMA_MODEL))
        print("Response words : {}".format(len(response.split())))
        print("Sections found : {} of {}".format(len(sections), len(HEADINGS)))
        print(utils.DASH)
        print(response)
        print(utils.DASH)
    return response
```

Follow-up questions (question_module.py)

```
def generate(verbose=True):
    """Q6 - generate follow-up questions for the reported symptoms."""
    state = utils.load_state()
    if not state.get("symptom_text"):
        print("Please enter the symptoms first (option 1).")
        return None

    prompt = ("Reported symptoms:\n{}\n\nInformation not provided:\n{}\n\n"
              "Details usually relevant for symptoms of this kind:\n{}\n\n"
              "Write five follow-up questions.").format(
        "\n".join("- {}".format(s) for s in state.get("symptoms", [])) or
state["symptom_text"],
        "\n".join("- {}".format(m) for m in state.get("missing", [])) or "- none",
        "\n".join("- {}".format(c) for c in state.get("usually_checked", [])[:6]) or "-
none")

    questions = clean(utils.ask(SYSTEM, prompt, temperature=0.3))
    utils.update_state(questions=questions)

    if verbose:
        print("FOLLOW-UP QUESTIONS")
        print(utils.DASH)
        for n, q in enumerate(questions, 1):
            for i, line in enumerate(utils.wrap(q, 56)):
                print("{:>3}. {}".format(n, line) if i == 0 else "     {}".format(line))
        print(utils.DASH)
        print("Questions generated : {}".format(len(questions)))
        print("Model               : {}".format(utils.OLLAMA_MODEL))
        print(utils.DASH)
    return questions
```

Medical attention guidance and disclaimer (safety_module.py)

```
def guidance_lines():
    """The medical-attention wording for the current case."""
    lines = utils.wrap(GENERAL, 56)
```

```
    if flagged_terms():
        lines += [""] + utils.wrap(ESCALATION, 56)
    state = utils.load_state()
    severity = str(state.get("severity", "Not provided"))
    lines += ["",
              "Severity reported by the user : {}".format(severity),
              "This system does not make a clinical triage decision."]
    return lines


def guidance(verbose=True):
    """Q7 - display the medical attention guidance."""
    if not utils.require("symptom_text", "Please enter the symptoms first (option 1)."):
        return None
    lines = guidance_lines()
    utils.update_state(guidance=lines, urgent_terms=flagged_terms())

    if verbose:
        print("MEDICAL ATTENTION")
        print(utils.DASH)
        for line in lines:
            print(line)
        print(utils.DASH)
        terms = flagged_terms()
        print("Urgent terms detected in the description : {}".format(
            ", ".join(terms) if terms else "none"))
        print(utils.DASH)
    return lines
```

Saving the report (save_module.py)

```
def save(verbose=True):
    """Q10 - save the generated analysis to outputs/medical_analysis.txt."""
    state = utils.load_state()
    if not state.get("sections"):
        print("Please analyze the symptoms first (option 3).")
        return None

    report = state.get("structured") or analysis_module.structured(verbose=False)

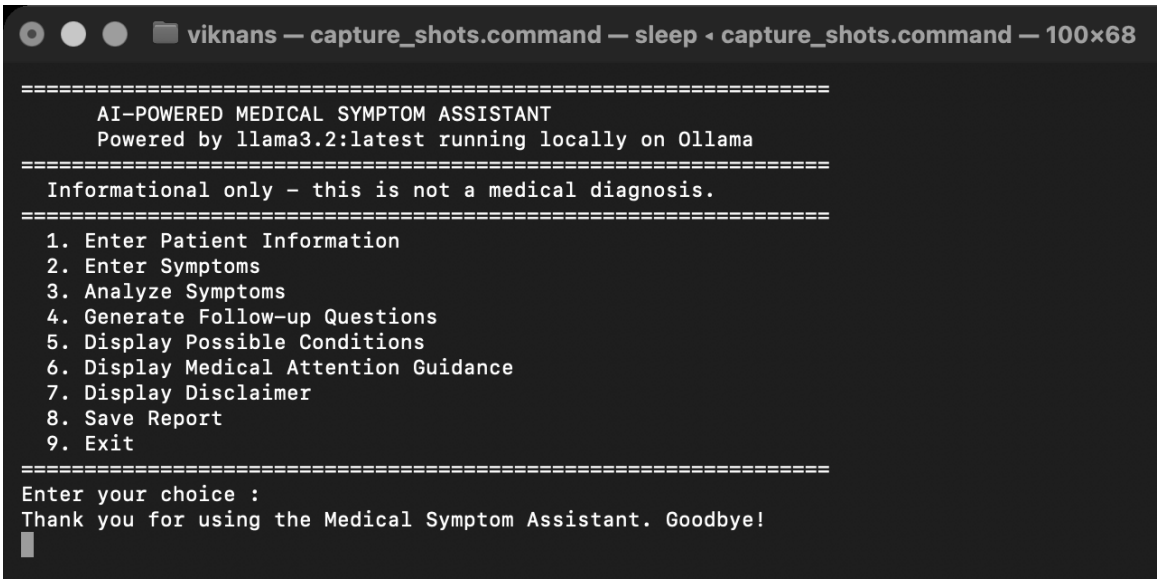
    lines = [
        "AI-POWERED MEDICAL SYMPTOM ASSISTANT - ANALYSIS REPORT",
        "=" * 55,
        "Date  : {}".format(datetime.datetime.now().strftime("%d/%m/%Y %H:%M")),
        "Model : {}".format(utils.OLLAMA_MODEL),
        "=" * 55,
        "",
        report,
        "",
        "RAW MODEL RESPONSE",
        "-" * 55,
        state.get("response", ""),
        "",
    ]

    try:
        os.makedirs(utils.OUTPUT_DIR, exist_ok=True)
        with open(utils.ANALYSIS_FILE, "w") as f:
            f.write("\n".join(lines))
    except OSError as exc:
        raise utils.ModelError("Could not save the analysis: {}".format(exc))
```

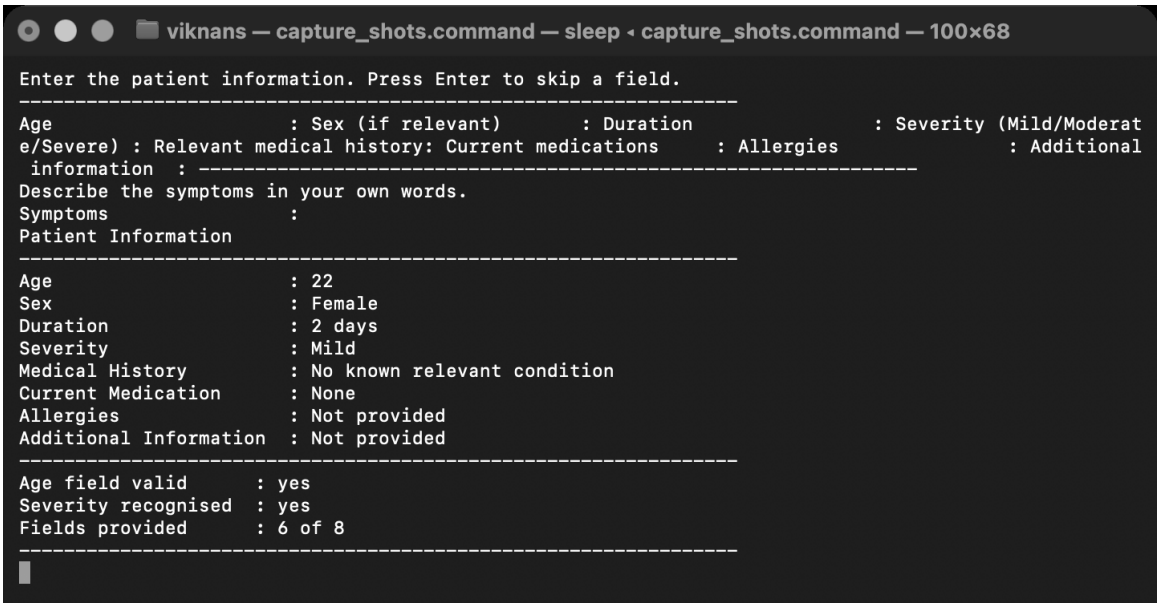
GitHub Link: https://github.com/ViknanS/exp6-AI-Powered-Medical-Symptom-Assistant.git

LinkedIn Post Link: https://www.linkedin.com/in/viknan/

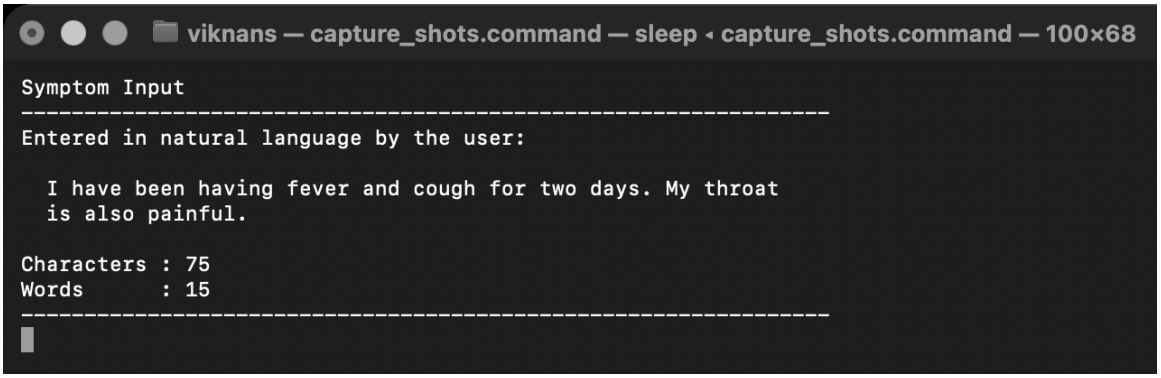
Output Screenshots:



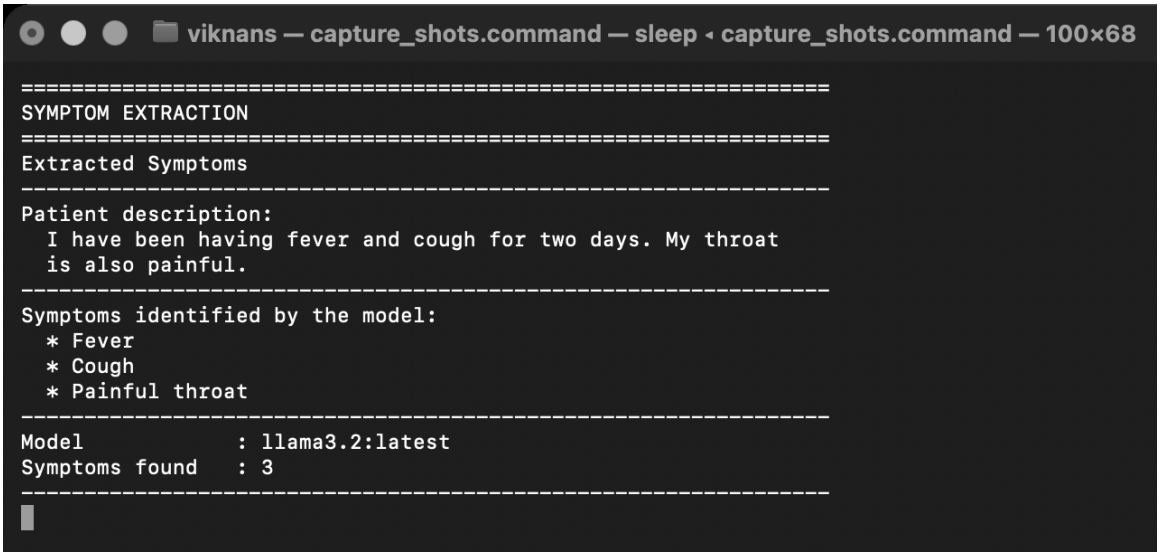
Home Screen



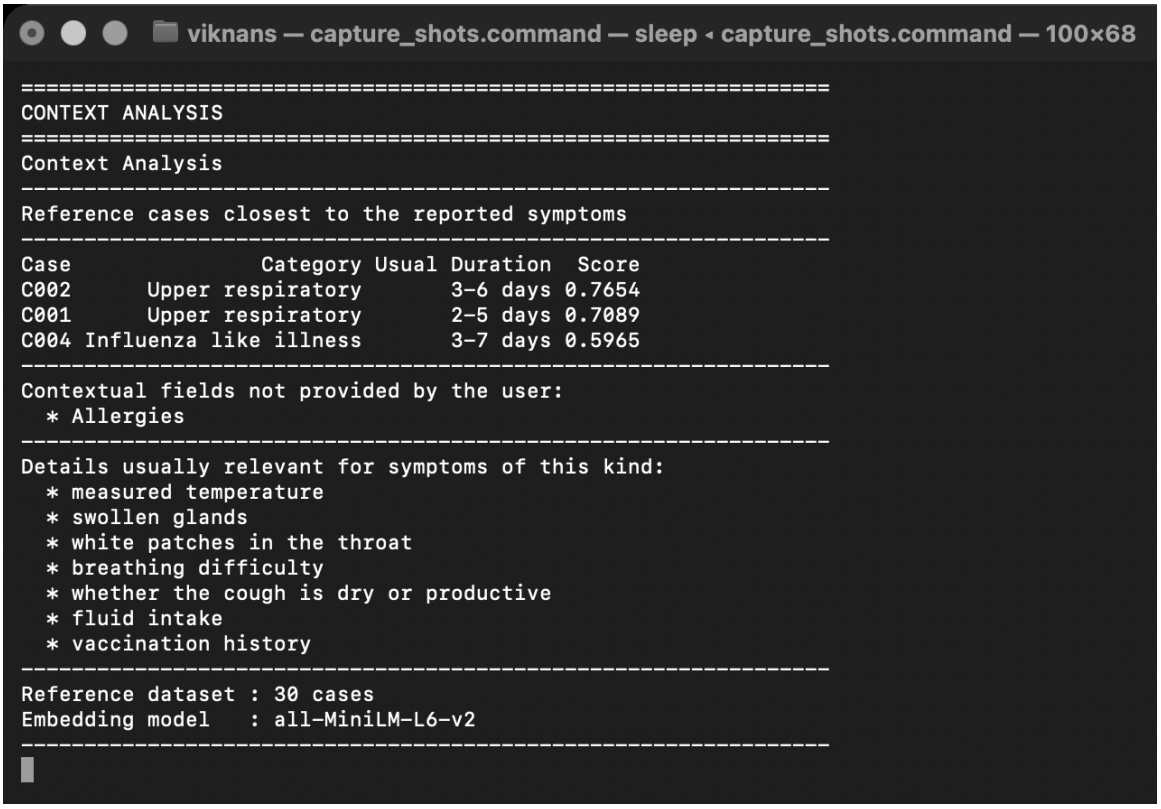
Patient Information Input



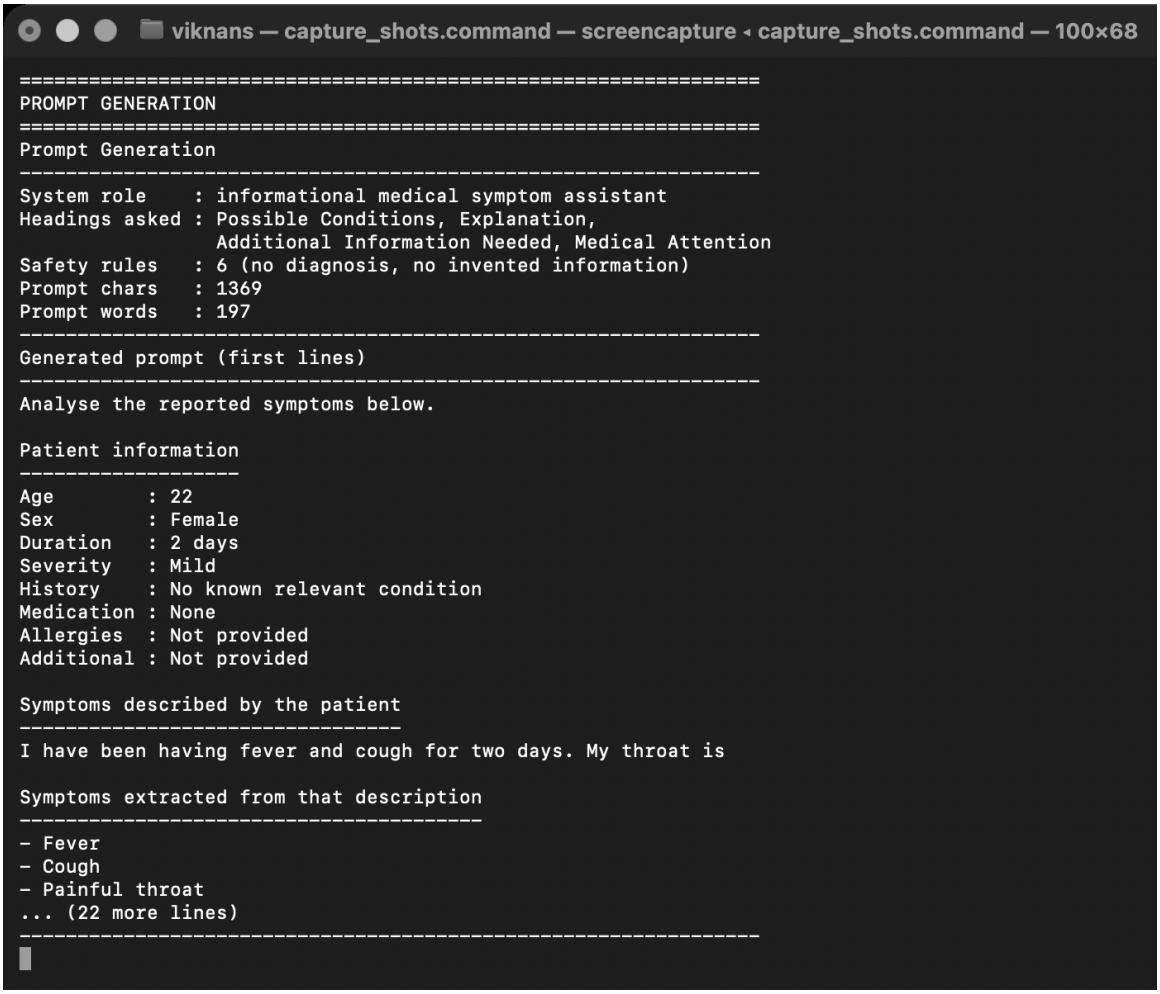
Symptom Input



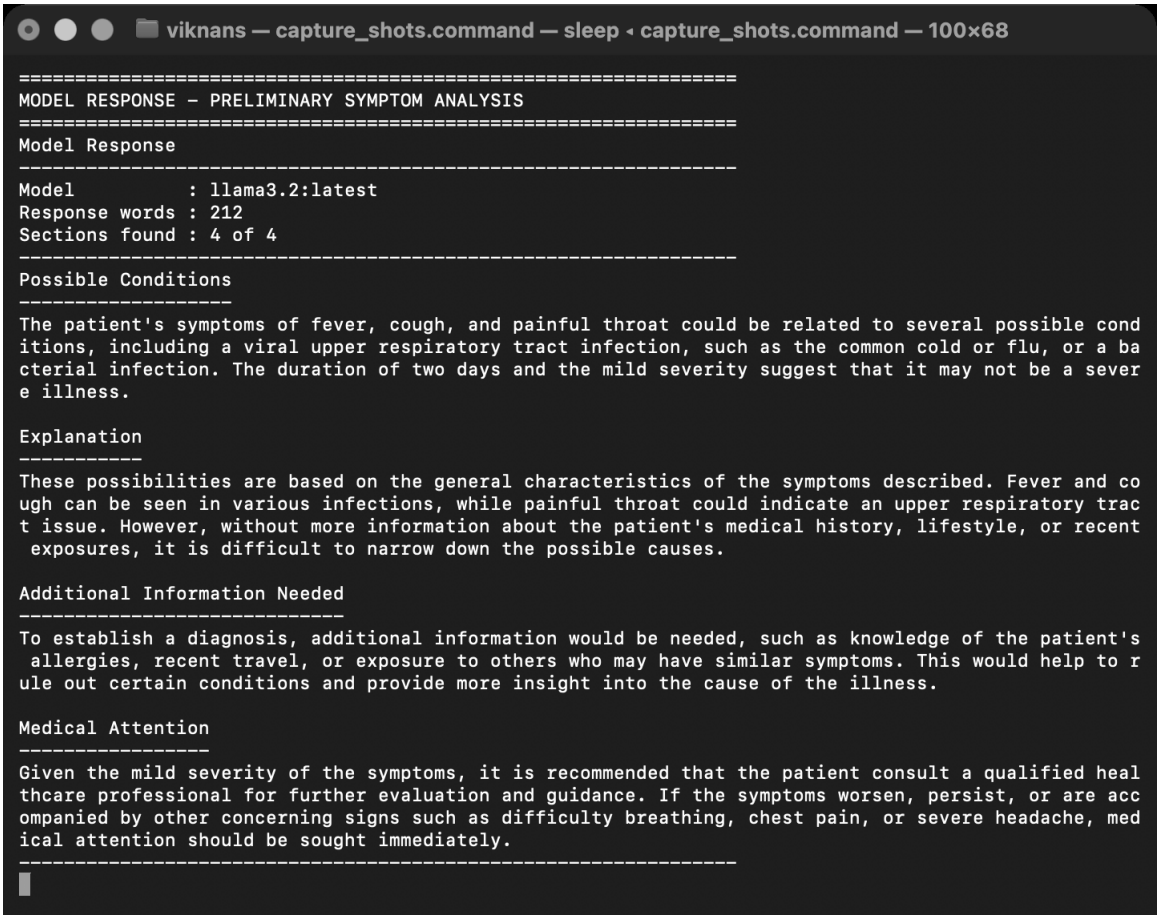
Extracted Symptoms



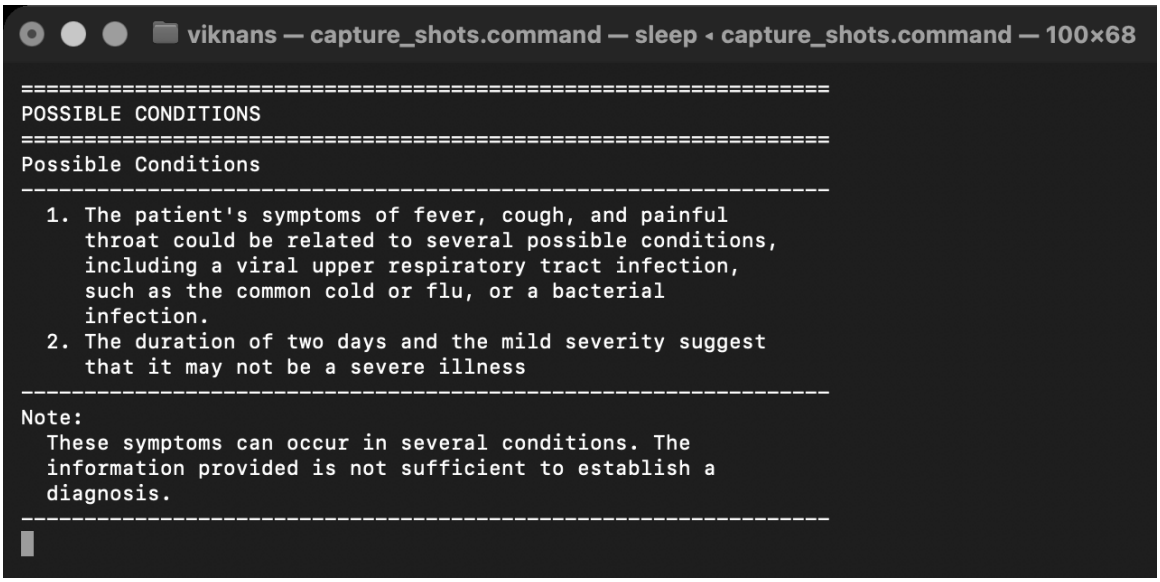
Context Analysis



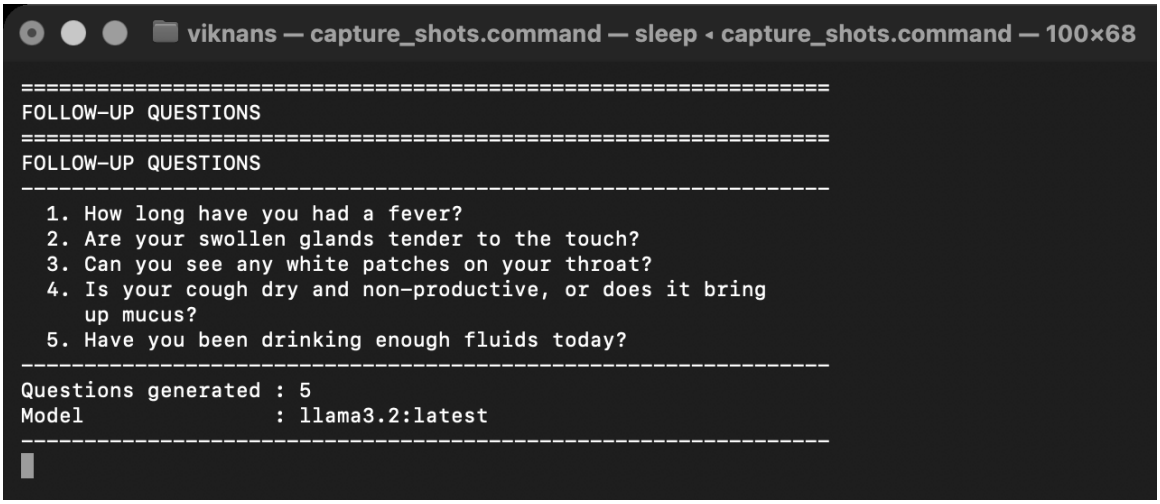
Prompt Generation



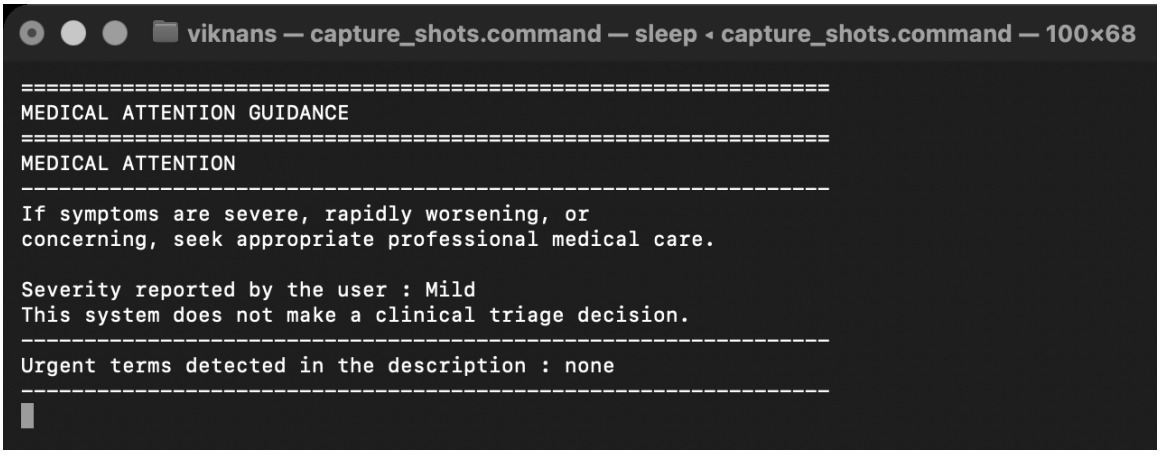
GPT-4 Response



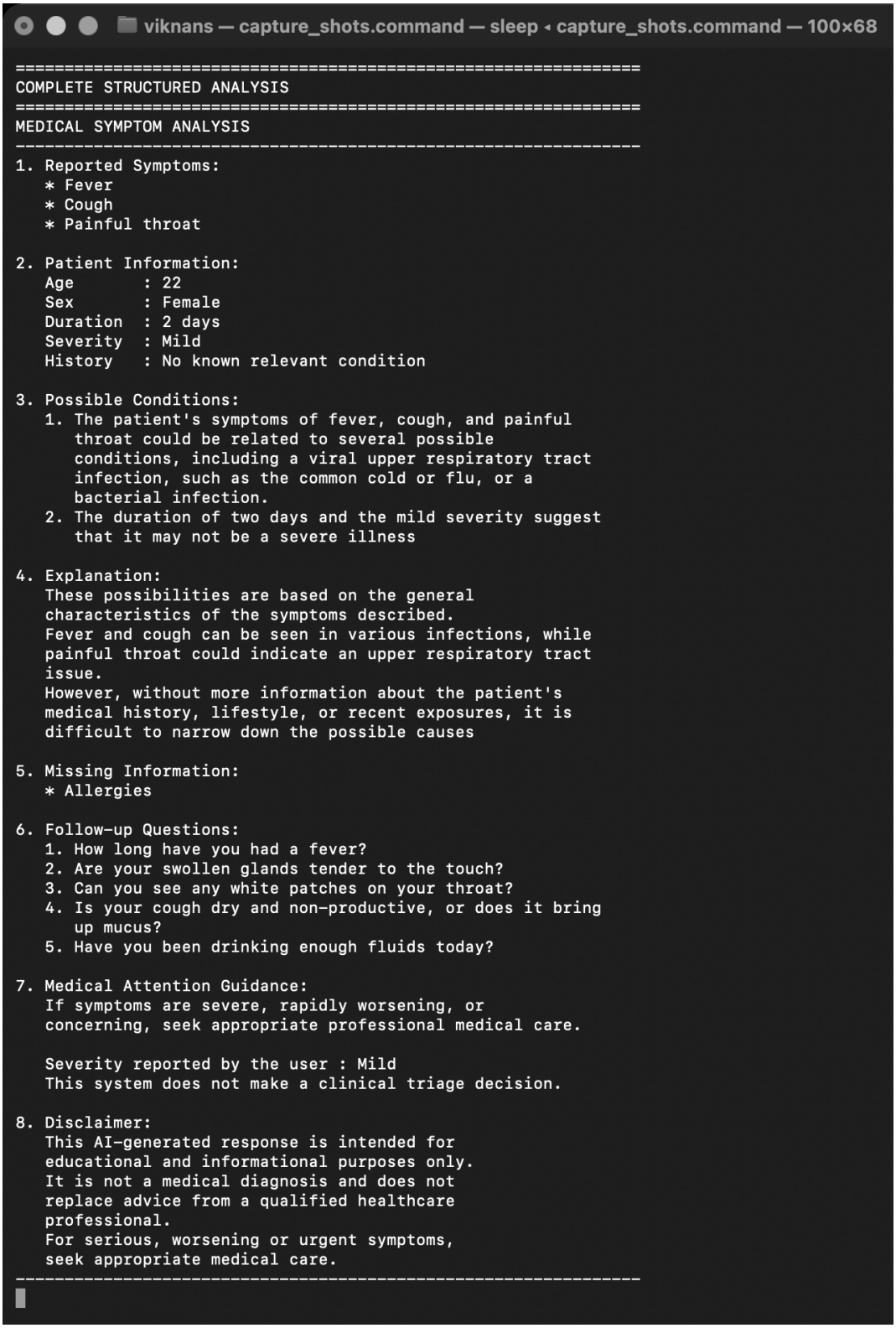
Possible Conditions



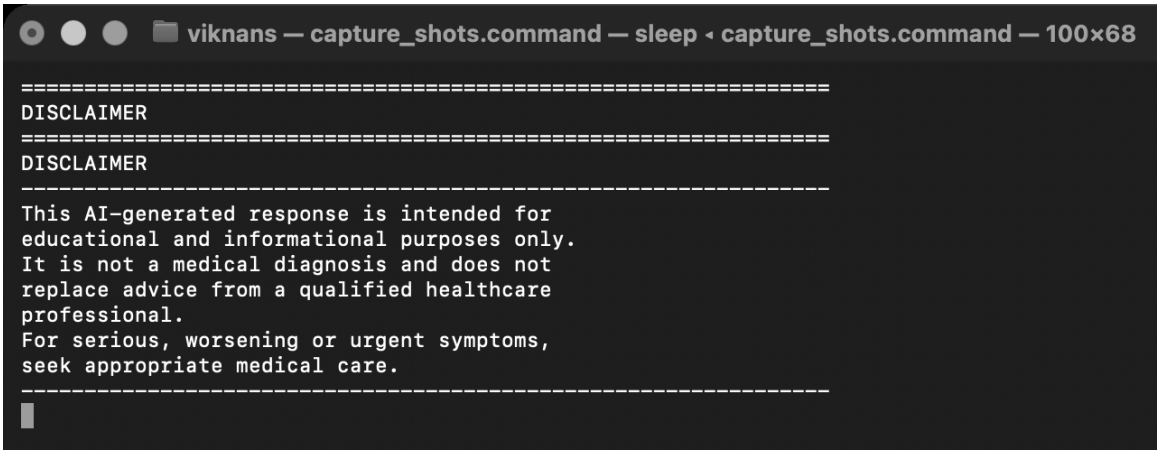
Follow-up Questions



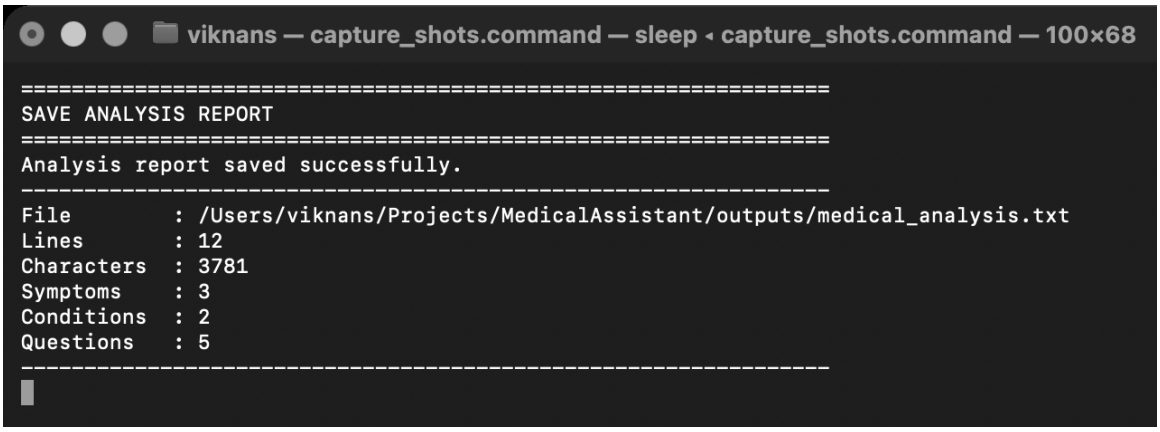
Medical Attention Guidance



Complete Structured Analysis



Disclaimer



Saved Report

Result:

Thus, an AI-powered medical symptom assistant using GPT-4 was successfully developed. The system accepts user-reported symptoms and relevant contextual information, performs preliminary symptom analysis, generates possible conditions and follow-up questions, provides appropriate medical-attention guidance and produces a structured informational response with a safety disclaimer.